Contents

BATCH PROGRAMMING GUIDE 3

PART 1: FOUNDATIONAL TOPICS (BEGINNER → INTERMEDIATE) 3

Limitations: When to Level Up 8

Creating Batch Files 9

Saving with Correct Encoding (ANSI vs UTF-8) 10

Running Batch Files (Multiple Ways) 11

Execution Policies (None for Batch, Unlike PowerShell) 13

Comments and Documentation 14

Documentation Best Practices 17

Echo Command (Output) 18

Pause and User Interaction 21

BATCH PROGRAMMING GUIDE

PART 1: FOUNDATIONAL TOPICS (BEGINNER → INTERMEDIATE) 💻📜🔥

What is a batch file? (.bat vs .cmd)

Welcome to the world of batch scripting! 🎉 This is where you'll learn to talk directly to Windows using its own language—simple, text-based commands that can save you tons of time and automate repetitive tasks. No defensive and offensive malware with cmd, no scary stuff—just pure, honest-to-goodness automation.

Think of a batch file as a **to-do list for your computer**.

You write down a series of commands that you'd normally type one by one in the Command Prompt, save them in a plain text file, and then—with a double-click—Windows runs them all in order.

It's like having a little robot that does the typing for you.

Batch files have been around since the MS-DOS days, and they're still kicking today because they're simple, fast, and get the job done. They're perfect for:

- Cleaning up temporary files 🧹
- Backing up important folders 💾
- Opening a bunch of programs at once 🚀
- Automating repetitive admin tasks ⚙️

😊 The Two Flavors: .bat vs .cmd

You'll see batch files with either a .bat or a .cmd extension. Are they different? Yes and no. Let's break it down:

FILE_EXTENSION_TELEMETRY: BATCH_FLAVORS

.bat

LEGACY STANDARD

The Original. Born in the MS-DOS days. It is the most recognized extension. When you run this, Windows uses its "Classic" command processing rules.

.cmd

NT ARCHITECTURE

The Professional. Introduced for Windows NT. It was built specifically for `cmd.exe`. It handles internal errors and environment changes with more precision.

🔍 **The Real Difference:** It all comes down to ERRORLEVEL. In a .bat file, if a command works, it doesn't always reset the error counter. In a .cmd file, every command (even a successful one) updates the error status. This makes .cmd more reliable for complex automation.

The Real Difference: Error Handling

FEATURE	.BAT (LEGACY)	.CMD (NT LOGIC)
Error Reset	Only updates on **failure**.	Updates on **every** command.
Success State	Retains old error codes if current command passes.	Resets `ERRORLEVEL` to 0 instantly.
Logic Reliability	Risky for deep conditional nesting.	Preferred for modern automation.
Internal Commands	Often ignores `SET` success status.	Strictly reports status of all internal commands.

📜 Creating Your First Batch File (No Coding Experience Needed!)

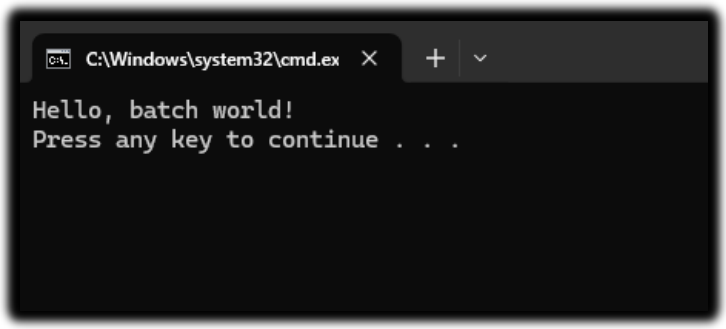
1. **Open Notepad** (or any text editor—just not Word!).
2. Type this:

```
@echo off
echo Hello world from batch!
pause
```

1. **Save the file** with a name like hello.bat. Make sure you choose "All Files" as the type, not ".txt".
2. **Double-click it** in File Explorer.

What happens?

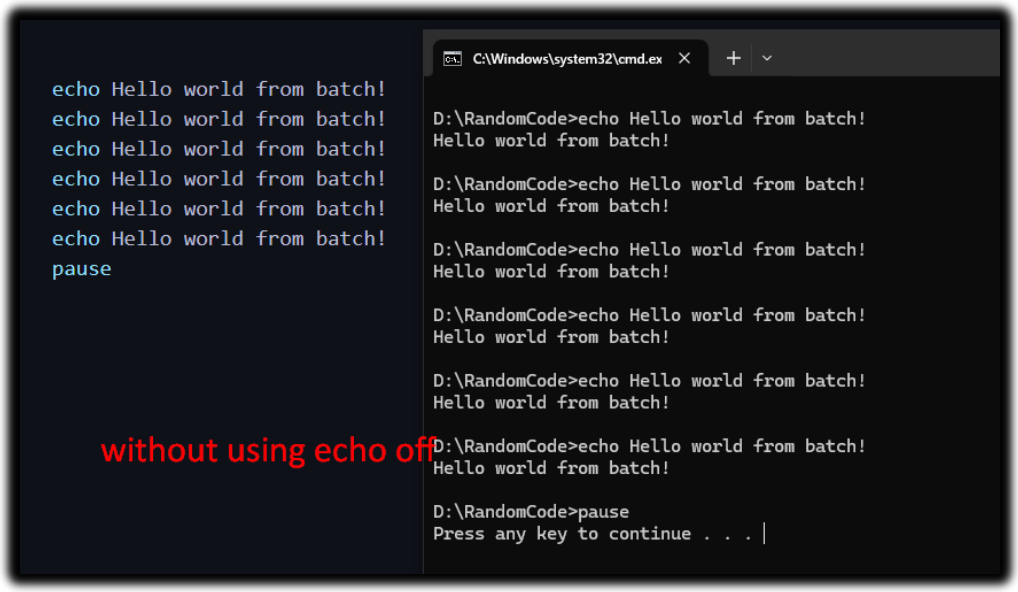
- A black command window pops up.
- It says "Hello, batch world!"
- Then it says "Press any key to continue . . ."
- You press a key, and the window closes.



Congratulations! You just ran your first batch script. 🎈

👀 What Those Commands Mean

COMMAND	EXECUTION & LOGIC
`@echo off`	Silences the command prompt display. It prevents the script from printing every line of code before executing it. *Result: A clean, professional user interface.*
`echo [text]`	The primary output command. It displays strings or variables directly to the user's console window. *Result: Communication between the script and the user.*
`pause`	Suspends script execution and waits for a keyboard interrupt. Prevents the CMD window from closing automatically. *Result: Keeps the output visible until the user is finished.*



🖥️ Running Batch Files: The Many Ways

- **Double-click** in File Explorer – runs in a new window (which closes when done).
- **Type the filename** in Command Prompt while in the same folder – runs in that same window.
- **Drag & drop** the file onto the Command Prompt window – pastes the path, then press Enter.

🤔 Why Learn Batch in 2026? Isn't PowerShell Better?

Great question! ✅ PowerShell is indeed more powerful—it's object-oriented, has access to .NET, and can do amazing things. But batch scripting is still valuable because:

- **It's everywhere** – Every Windows machine has CMD. No extra installs, no execution policy hassles.
- **It's fast** – For quick, simple tasks, firing up a batch file is instant.
- **It's the foundation** – Understanding batch helps you understand how Windows works under the hood.
- **Legacy scripts** – You'll encounter them. Knowing batch means you can read, fix, or improve them.

Think of batch as your **entry-level automation tool**. Later you can level up to PowerShell if you need more power. But for now, let's enjoy the simplicity! 😄

💻 How Batch Scripts Execute (Line-by-Line Interpretation)

Batch files are like a **recipe card** for your computer. When you run a .bat or .cmd file, here's what happens behind the scenes:

- **The Command Interpreter (CMD.EXE)** reads the file from the very first line to the very last, one line at a time. 📖
- Each line is treated as a command—just as if you'd typed it yourself in the Command Prompt.
- The interpreter executes the command, waits for it to finish, then moves to the next line.
- If a command fails, the script **keeps going** by default (unless you've added error checking). This is both a feature and a danger!
- Once the last line is done, the script exits, and the Command Prompt window closes (unless you used pause).

Key points to remember:

- No compilation—it's pure **interpretation**. What you write is what gets executed, instantly.
- It's **sequential**—commands don't run in parallel unless you explicitly use start.
- Variables are expanded as the line is read, which can lead to tricky behavior in loops (but we'll cover that later).

Analogy: Think of it as handing a list of chores to a very literal assistant. They'll do chore #1, then #2, then #3, and stop when the list ends—even if something goes wrong halfway. 📜

✨ Use Cases: Where Batch Files Shine

Batch scripting isn't dead—it's just specialized. Here's where it's still a go-to tool in 2026:

I. Automation (The Everyday Hero)

- **Repetitive file tasks**: Clean up temporary folders, rename hundreds of files in a pattern, or copy specific documents to a backup drive.
- **Scheduled jobs**: Use Task Scheduler to run a batch script every night—no GUI needed.
- **Launch multiple apps**: Open your daily toolkit (browser, email, editor) with one double-click.

II. System Administration (The IT Ally) 🛠️

- **Quick diagnostics**: A batch script can gather IP config, ping servers, and write results to a log file—all in seconds.
- **User management**: Reset local user passwords or add network drives during login.
- **Startup/shutdown scripts**: Deploy policies or clean temp files when machines boot or shut down.

III. Penetration Testing (The Ethical Hacker's Sidekick) 🕵️

- **Why batch?** Because it's on **every Windows machine**—no extra tools needed. A simple script can enumerate users, check open shares, or dump system info.
- **Persistence**: Attackers (and pen testers) often use batch scripts to maintain access or automate reconnaissance.
- ⚠️ **Important**: This is for educational and authorized testing only. Using batch for malicious purposes is illegal and unethical. Always get permission.

Limitations: When to Level Up 📈

Batch 🆚 PowerShell, Python and Compiled Languages (C, C++, C#)

COMPARISON PAIR	THE KEY DIFFERENCE	LIMITATIONS OF BATCH
CMD vs PowerShell	Text vs. Objects	Batch only sees text. PowerShell sees the "DNA" of a file (size, owner, date) as a single package.
CMD vs Python	Windows vs. Universal	Batch stops at the Windows border. Python works on Macs, Linux, and even refrigerators.
CMD vs C++/C#	Script vs. Software	Batch is slow and has no safety net. Compiled languages are built for speed and massive apps.

Modern Perspective (2026)

Batch scripting is **not obsolete**—it's a **specialist tool**. Think of it like a hammer: great for nails, but you wouldn't build a house with just a hammer. Use it for:

- Quick one-off tasks
- Environments where you can't install extra software
- Legacy system maintenance
- Learning the fundamentals of how Windows works

When your needs grow, you'll naturally reach for PowerShell, Python, or compiled languages. But for now, enjoy the simplicity and power of batch! 🛠️

Creating Batch Files

Alright, let's get our hands dirty! 👐 This is where you stop reading and start **doing**. We'll cover how to create batch files, save them correctly, and run them in all the different ways Windows allows.

A batch file is just a plain text file with a special extension. You can create one with any text editor—even the simplest ones work fine. Here are your options:

📝 Notepad (The Classic)

- **Where to find it**: Start menu → type "Notepad"
- **Pros**: It's on every Windows machine. No install needed. Zero distractions.
- **Cons**: No syntax highlighting, no line numbers, no auto-completion. You're flying blind.
- **Best for**: Quick one-liners or when you're on a locked-down machine.

💻 Notepad++ (The Upgrade)

- **Where to get it**: notepad-plus-plus.org (free, open source)
- **Pros**: Syntax highlighting for batch! Colors make it way easier to spot mistakes. Line numbers, tabbed editing, search/replace across files.
- **Cons**: You have to download it (but seriously, why don't you have this already? 😄)
- **Best for**: Everyday batch scripting. This is the sweet spot.

🧑💻 VS Code (The Powerhouse)

- **Where to get it**: code.visualstudio.com (free)
- **Pros**: IntelliSense (auto-completion), debugging, Git integration, extensions for batch syntax, runs anywhere.
- **Cons**: Overkill if you're just writing simple batch files. Takes longer to load.
- **Best for**: When you're already using VS Code for other coding, or when your batch scripts grow into something bigger.

🖥️ Command Prompt Itself (The Old-School Way)

- **How**: copy con myfile.bat then type your lines, then press Ctrl+Z then Enter.
- **Pros**: No editor needed at all—pure command line.
- **Cons**: Painful to edit mistakes. Don't do this unless you're in a recovery environment.

My recommendation: Start with **Notepad++ or VScode**. It's lightweight, free, and makes batch files readable. Later, if you're also learning Python or web dev, VS Code can handle everything.

Saving with Correct Encoding (ANSI vs UTF-8)

This sounds boring, but **it matters**. Batch files are picky about how they're saved. Save them wrong, and you'll get weird errors or gibberish characters.

🧪 ANSI (The Safe Choice)

- **What it is**: The old Windows character set (CP-1252 for Western languages).
- **Why it works**: CMD.EXE was built in the ANSI era. It expects ANSI by default.
- **Best for**: Simple scripts with plain English text and no special symbols.

🌍 UTF-8 (The Modern Standard)

- **What it is**: Unicode encoding that handles every language and symbol on the planet.
- **The problem**: CMD.EXE doesn't natively understand UTF-8 unless you explicitly tell it to with the chcp 65001 command. Even then, it can be flaky.
- **The BOM problem**: If you save UTF-8 with a BOM (Byte Order Mark) in Notepad, CMD will read those first three bytes as garbage and throw an error. 😫

✅ The Golden Rule for Batch Files

- **Stick to ANSI** unless you absolutely need international characters.
- If you must use UTF-8:
  - Save **without BOM** (Notepad++ and VS Code can do this; regular Notepad cannot).
  - Add chcp 65001 >nul as the first line of your script to switch the console to UTF-8.
  - Test thoroughly—some commands still misbehave.

How to set encoding in your editor:

📄 DATA_ENCODING_PROTOCOL

TARGET: ANSI / WINDOWS-1252

TARGET EDITOR	PROTOCOL FOR ANSI EXPORT
Notepad	File → Save As → Encoding: [ANSI]
Notepad++	Encoding → Character sets → Western European → Windows-1252
VS Code	Status Bar (UTF-8) → Save with Encoding → Pick [Windows 1252]

🚀 Bottom line: When in doubt, save as ANSI. It works everywhere, every time.

Running Batch Files (Multiple Ways)

You've got options—lots of them. Here's when to use each method.

🖱️ Double-Click in File Explorer (The Easy Way)

- **What happens**: Opens a new Command Prompt window, runs the script, and **closes the window when done**.
- **Best for**: Quick runs where you don't need to see the output afterward.
- **Warning**: If the script errors out, the window vanishes instantly—you won't see what went wrong. That's why we use pause in examples!

💻 From an Existing Command Prompt (The Developer Way)

1. Open CMD (type cmd in Start menu).
2. Navigate to your script's folder using cd (e.g., cd C:\MyBatchFiles).
3. Type the filename: myfile.bat and press Enter.

- **What happens**: Runs in the current window. Output stays visible. You can see errors.
- **Best for**: Testing and debugging.

🚀 Drag-and-Drop (The Lazy Way)

- Drag your .bat file from File Explorer into an open Command Prompt window. The full path gets pasted. Press Enter.
- **Best for**: When you're already in CMD but don't want to type long paths.

⏰ Task Scheduler (The "Set It and Forget It" Way)

- **What it does**: Runs your batch file automatically at a specific time (daily, weekly, at login, etc.).
- **How to set up**:
    1. Open Task Scheduler (type "Task Scheduler" in Start).
    2. Click "Create Basic Task".
    3. Follow the wizard: give it a name, choose trigger (e.g., "Daily"), choose action "Start a program", browse to your .bat file.
    4. Done! It'll run on schedule whether you're logged in or not.
- **Best for**: Backup scripts, cleanup routines, automated reports.

🔄 As Part of Other Scripts

- You can call a batch file from **inside another batch file** just by writing its name.
- You can also call it from PowerShell, Python, or any language that can execute external commands.

Execution Policies (None for Batch, Unlike PowerShell)

If you've used PowerShell, you know about **execution policies**—those annoying messages when you try to run a script you downloaded. PowerShell blocks scripts by default for security.

Batch files have no such protection. 🛡️

- A .bat or .cmd file will run **no matter where it came from**—email, USB stick, sketchy website, it doesn't care.
- No prompts, no warnings, no "are you sure?" It just executes.

⚠️ This Is Both a Feature and a Danger

- **Feature**: Simple, fast, no hoops to jump through.
- **Danger**: If you double-click a malicious batch file, it can do real damage instantly—delete files, change settings, spread to network drives.

✅ Safe Habits for Batch Files

1. **Never run a batch file from an untrusted source.** Treat them like executables (.exe).
2. **Always inspect the code first.** Right-click → Edit, or open in Notepad. Read every line. If you don't understand it, don't run it.
3. **Run unknown scripts in a sandbox** (virtual machine) if you must test them.
4. **Back up your data.** Always.

Remember: With great power comes great responsibility. You're the gatekeeper. 👮

Modern Perspective (2026)

Creating and running batch files is **easier than ever**, thanks to modern editors and Task Scheduler. The process hasn't changed in decades—and that's the beauty of it.

- **Editors**: Notepad++ or VS Code make batch files readable with syntax coloring.
- **Encoding**: ANSI is still the safe bet, but UTF-8 with BOM-free saving is now possible.
- **Running**: Double-click for quick tasks, CMD for testing, Task Scheduler for automation.
- **Security**: No execution policy means you're responsible. Stay vigilant.

Comments and Documentation

Comments are like sticky notes you leave for yourself (or others) inside your code.

They don't do anything—they're just there to explain what's going on.

And trust me, future-you will thank present-you for writing them! 😊

Batch gives you two ways to write a comment that lasts only one line:

🤫 REM (The Classic)

REM stands for "remark." Anything after REM is ignored by the command interpreter.

```
REM This is a comment. The computer ignores it.
echo Hello World   REM You can also put comments at the end of a line!
```

:: (The Sneaky Shortcut)

:: is a hack that uses the label syntax (:label) but with an invalid character, so it's treated as a comment.

It's faster to type, but it has a quirk: **it can cause errors inside parenthesized blocks** (like in if statements or for loops).

So, REM is safer in those situations.

```
:: This is also a comment, but be careful where you use it!
```

Which one should you use?

- For quick, everyday comments: :: is fine.
- Inside code blocks (if, for), stick with REM.
- Many batch pros use REM everywhere to avoid surprises.

Multi-Line Comments (The GOTO Workaround)

Batch doesn't have a built-in way to write multi-line comments. But clever folks found a workaround using GOTO:

📂 SCRIPT_EXECUTION_FLOW

```
@echo off
GOTO :comment_block
  This entire block is ignored!
  You can write as many lines as you want.
  Even with special characters *&^%$#@!
  None of this will execute because the computer
  just "jumped" over it to the next label.
:comment_block
echo The real code starts here.
set /a result=10 + 5
echo Calculation complete: %result%
pause
```

💡 The Teleport Analogy:

Think of GOTO like a **Teleporter**.

The script is walking down the street (the lines of code), hits the teleporter at the top, and instantly blinks to the exit pad at the bottom.

It never actually walks past the text in the middle, so it never sees it.

💡 The Breakdown:

- **:: (The Double Colon):** This is a **Single Line Comment**. It tells the computer: *Ignore everything on this specific line.* You have to put it at the start of every single line you want to hide.
- **:comment_block (The Label):** This is a **Label,** we shall cover later. Think of it like a **Destination Marker** or a Landing Pad. By itself, a label doesn't do anything, but the GOTO command uses it to jump around.

How it works:

- GOTO :comment_block jumps to the label :comment_block, skipping everything in between.
- The lines between are never interpreted—they're effectively a multi-line comment.
- Just make sure your label name isn't used elsewhere in the script.

Caveat: This method skips the lines, but they're still loaded into memory. For huge blocks, it's inefficient. But for a dozen lines? Totally fine.

Documentation Best Practices

Good comments turn a cryptic script into a readable story. Here's how to do it right:

📌 Header Comments (The "What & Why")

At the top of every batch file, include a header like this:

```
@echo off
REM ============================================
REM Script:   backup.bat
REM Author:   Your Name
REM Date:     2026-02-14
REM Purpose:  Backs up Documents folder to D:\Backups
REM           Logs results to backup.log
REM ============================================
```

📝 Inline Explanations (The "How")

Explain tricky parts, but don't state the obvious:

```
REM Good:
setlocal enabledelayedexpansion   REM Allows variables to update inside loops

REM Bad (too obvious):
echo Starting backup   REM This prints a message
```

🧹 Comment As You Go

Don't write comments after the fact—you'll forget what you did. Write them while you're coding. Future-you will be grateful! 💖

Echo Command (Output)

The echo command is your script's voice. It's how you talk to the user, show progress, or reveal what's happening under the hood.

Displaying Text: ECHO Hello World

The simplest form:

```
REM Bad (too obvious):
echo Starting backup   REM This prints a message

echo Hello, world!
```

This prints Hello, world! to the console. You can put any text after echo, and it'll appear on screen.

Empty echo (just echo with nothing after) prints the current echo setting (ECHO is on or ECHO is off). Not super useful, but good to know.

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Rennex>echo
ECHO is on.

C:\Users\Rennex>echo off
echo hello world
hello world

echo on

C:\Users\Rennex>echo
ECHO is on.
```

Suppressing Command Output: @ECHO OFF

By default, batch shows every command as it runs (called "echoing").

That's noisy! I showed it in the previous images.

Most scripts start with **@echo off**

- echo off turns off command echoing for the rest of the script.
- The @ at the beginning hides that line itself, so you don't see echo off either.

Now your script runs silently—only the output you explicitly echo appears. Clean and professional. ✨

Blank Lines: ECHO. or ECHO:

Sometimes you need an empty line for spacing. Use echo followed immediately by a dot (.) or colon (:):

You can see when I put **echo. hello** it places a space before hello begins.

🎩 The Escape Hat: ^

Put a caret (^) right before the character to print it literally:

```
echo This is a caret: ^^
echo This is a pipe: ^|
```

Without the caret, the pipe would be interpreted as a command connector.

💬 Quotes for Safety

If you're echoing a long string with special characters, wrap the whole thing in double quotes:

```
echo "This & that | the other thing"
```

But note: the quotes themselves will appear in the output.

To avoid that, use ^ escapes for individual characters.

🧪 Example: Printing a Copyright Symbol

```
C:\Users\Rennex>echo Copyright ^© 2026
Copyright © 2026
```

(Note: © isn't a special character, but it's a good example of a non-ASCII symbol—make sure your file is saved in ANSI or UTF-8 without BOM!)

Pause and User Interaction

Batch files can ask the user to wait or even pause for input. Here's how to make your scripts interactive.

PAUSE Command (Wait for Keypress)

The classic pause does exactly what it says: stops everything and waits for the user to press any key.

```
echo This will show...
pause
echo ...and this will show after you press a key.
```

By default, it displays: Press any key to continue . . . and waits.

Why use it?

- To let the user read output before the window closes.
- To give them a chance to cancel (by pressing Ctrl+C).
- To create a simple "press any key to continue" step in a script.

Custom Pause Messages

You can't directly change pause's message, but you can fake it:

```
echo Ready to proceed? Press any key...
pause > nul
```

🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🟢🔴🔴🔴🔴🔴🔴🔴🔴🔴🔴🔴🔴🔴

1.3 Comments and Documentation

- Single-line comments (REM, ::)
- Multi-line comments (GOTO workaround)
- Documentation best practices (header comments, inline explanations)

1.4 Echo Command (Output)

- Displaying text (ECHO Hello World)
- Suppressing command output (@ECHO OFF)
- Blank lines (ECHO. or ECHO:)
- Special characters (escaping with ^, quotes)

1.5 Pause and User Interaction

- PAUSE command (wait for keypress)
- Custom pause messages
- TIMEOUT command (auto-continue after N seconds)

1.6 Exit Codes and Error Levels

- What are exit codes? (0 = success, non-zero = error)
- Setting exit codes (EXIT /B <code>)
- Checking ERRORLEVEL
- Common exit codes (0, 1, 2, etc.)

MODULE 2: VARIABLES (Data Handling)

2.1 Environment Variables (Built-in)

- System variables (%PATH%, %USERPROFILE%, %TEMP%, %COMPUTERNAME%, %USERNAME%)
- Date/time variables (%DATE%, %TIME%)
- Special variables (%CD%, %RANDOM%, %CMDCMDLINE%)
- Listing all environment variables (SET with no arguments)

2.2 User-Defined Variables

- SET command (SET var=value)
- No spaces around = (common pitfall)
- Reading variables (%var%)
- Unsetting variables (SET var=)

2.3 Variable Scope

- Global variables (persist in session)
- Local variables (SETLOCAL/ENDLOCAL)
- Nested SETLOCAL (scope stacking)

2.4 Delayed Expansion

- Why delayed expansion? (FOR loops, IF blocks)
- Enabling (SETLOCAL EnableDelayedExpansion)
- Syntax (!var! vs %var%)
- When to use each

2.5 Special Variable Syntax

- Command-line arguments (%0, %1, %2, ..., %9, %*)
- Batch file path/name (%~dp0, %~nx0, %~f0)
- Substring extraction (%var:~start,length%)
- String replacement (%var:old=new%)
- String length (workaround using loops or external tools)

2.6 Arithmetic Operations

- SET /A command (integer math only)
- Operators (+, -, *, /, %, &, |, ^, <<, >>)
- Precedence and parentheses
- Limitations (no floating-point, no large numbers >32-bit signed int)

MODULE 3: CONTROL FLOW (Logic and Loops)

3.1 IF Statements (Conditionals)

- Basic syntax (IF condition command)
- String comparison (IF "string1"=="string2")
- Case sensitivity (default case-sensitive, /I for case-insensitive)
- Numeric comparison (EQU, NEQ, LSS, LEQ, GTR, GEQ)
- File/directory existence (IF EXIST path)
- DEFINED check (IF DEFINED var)
- NOT operator (IF NOT condition)

3.2 IF...ELSE Statements

- ELSE clause syntax (must be on same line as closing parenthesis)
- Multi-line IF blocks (proper syntax)
- Nested IF statements

3.3 Error Level Checking

- IF ERRORLEVEL N (checks >= N, legacy)
- IF %ERRORLEVEL%==N (checks exact value, modern)
- Combining with commands (command && success_action || failure_action)

3.4 FOR Loops (Iteration)

- Basic FOR syntax (FOR %%var IN (set) DO command)
- Loop variable naming (%%i in scripts, %i in command line)
- Iterating over files/folders
- Wildcards (*.txt, ., etc.)

3.5 FOR /L (Numeric Range)

- Syntax (FOR /L %%i IN (start,step,end) DO command)
- Counting forwards (1 to 10)
- Counting backwards (10 to 1, negative step)
- Custom increments (count by 2, 5, etc.)

3.6 FOR /F (File Parsing)

- Syntax (FOR /F "options" %%i IN (source) DO command)
- Options breakdown:

    - delims= (delimiter characters, default space/tab)
    - tokens= (which columns to extract, 1-based)
    - skip= (skip first N lines)
    - eol= (end-of-line comment character, default ;)
    - usebackq (backticks for commands, quotes for files)
- Reading file line-by-line
- Parsing CSV files
- Processing command output

3.7 FOR /D (Directory Loop)

- Syntax (FOR /D %%i IN (set) DO command)
- Iterating over directories only (not files)
- Wildcards in directory paths

3.8 FOR /R (Recursive Loop)

- Syntax (FOR /R [path] %%i IN (set) DO command)
- Recursively searching subdirectories
- Finding all files of a type (*.log, *.txt)

3.9 Nested FOR Loops

- Multiple loop variables (%%i, %%j, %%k)
- Performance considerations (avoid deep nesting)
- Use cases (matrix operations, multi-dimensional processing)

3.10 GOTO and Labels

- Defining labels (:label_name)
- Jumping to labels (GOTO label_name)
- Infinite loops (GOTO loop, :loop, GOTO loop)
- Exit loops (GOTO :EOF or EXIT)

3.11 CALL Command (Subroutines/Functions)

- Calling labels as subroutines (CALL :label)
- Passing parameters to subroutines

- Returning values (ERRORLEVEL, environment variables)
- Calling external batch files

MODULE 4: FILE OPERATIONS (System Interaction)

4.1 Creating Files

- ECHO to file (ECHO text > file.txt)
- Appending (ECHO text >> file.txt)
- COPY command (copy files)
- TYPE command (display file contents)

4.2 Reading Files

- TYPE command (entire file)
- MORE command (paginated output)
- FOR /F (line-by-line processing)

4.3 Deleting Files

- DEL command (delete files)
- ERASE command (alias for DEL)
- Wildcards (*.tmp, *.log)
- Force deletion (/F, /Q flags)

4.4 Renaming and Moving Files

- REN/RENAME command (rename files)
- MOVE command (move files)
- Bulk renaming (FOR loop + REN)

4.5 Copying Files

- COPY command (simple copy)
- XCOPY command (advanced copy, subdirectories, attributes)
- ROBOCOPY command (robust copy, mirroring, restartable)

4.6 File Attributes

- ATTRIB command (view/modify attributes)

- Attributes: R (read-only), H (hidden), S (system), A (archive)
- Removing attributes (ATTRIB -H -R file.txt)

4.7 Directory Operations

- MKDIR/MD (create directory)
- RMDIR/RD (remove directory)
- CD/CHDIR (change directory)
- DIR (list directory contents)
- DIR options (/S, /B, /A, /O, etc.)

MODULE 5: STRING MANIPULATION (Text Processing)

5.1 Substring Extraction

- Syntax (%var:~start,length%)
- Positive indices (from left)
- Negative indices (from right)
- Omitting length (to end of string)

5.2 String Replacement

- Syntax (%var:old=new%)
- Replacing first occurrence
- Replacing all occurrences
- Removing characters (%var:old=%)

5.3 String Length

- No built-in command (workaround required)
- Method 1: Loop through characters
- Method 2: Use FINDSTR with regex
- Method 3: External tools (PowerShell, Python)

5.4 Concatenation

- Simple concatenation (SET result=%var1%%var2%)
- Building strings in loops

5.5 Case Conversion

- No built-in (workaround required)
- Method 1: FOR /F with external tools (PowerShell, VBScript)
- Method 2: Lookup table (manual A-Z mapping)

5.6 Trimming Whitespace

- Leading spaces (loop-based removal)
- Trailing spaces (loop-based removal)
- Both (combined approach)

MODULE 6: INPUT/OUTPUT (User Interaction)

6.1 Reading User Input

- SET /P command (SET /P var=Prompt:)
- Default values (if user presses Enter without input)
- Input validation (checking non-empty, numeric, etc.)

6.2 Displaying Output

- ECHO command (standard output)
- COLOR command (change console colors)
- TITLE command (set window title)
- CLS command (clear screen)

6.3 Redirection

- Output redirection (> file.txt)
- Append redirection (>> file.txt)
- Input redirection (< file.txt)
- Error redirection (2> error.txt)
- Combined (command > output.txt 2>&1)

6.4 Piping

- Pipe operator (command1 | command2)
- Chaining commands (output of one = input of next)

- Examples (DIR | FIND "txt", TASKLIST | FINDSTR "chrome")

6.5 Suppressing Output

- Redirect to NUL (command > NUL)
- Suppress errors (command 2> NUL)
- Suppress everything (command > NUL 2>&1)

6.6 ANSI Escape Codes (Color/Formatting)

- Enabling ANSI support (Windows 10+)
- Color codes (foreground, background)
- Cursor positioning
- Text formatting (bold, underline, etc.)

MODULE 7: FUNCTIONS AND SUBROUTINES (Code Reusability)

7.1 Defining Functions (Labels as Functions)

- Function syntax (:function_name)
- CALL to invoke (CALL :function_name)
- GOTO :EOF to return

7.2 Passing Parameters

- Using %1, %2, %3, etc. in function
- Shifting parameters (SHIFT command)
- Accessing all parameters (%*)

7.3 Returning Values

- Via ERRORLEVEL (EXIT /B <code>)
- Via environment variables (SET result=value)
- Via ENDLOCAL workaround (return across SETLOCAL boundary)

7.4 Local vs Global Scope

- SETLOCAL in function (local variables)
- ENDLOCAL to exit scope
- Returning values across SETLOCAL (tricks)

7.5 Recursive Functions

- Calling function from within itself
- Base case (termination condition)
- Use cases (tree traversal, factorial calculation)

MODULE 8: ERROR HANDLING (Robustness)

8.1 Understanding ERRORLEVEL

- What is ERRORLEVEL? (exit code of last command)
- Common values (0 = success, 1 = general error, 2 = file not found, etc.)
- Checking ERRORLEVEL (IF ERRORLEVEL 1, IF %ERRORLEVEL%==0)

8.2 Error Checking Patterns

- Check after every critical command
- Conditional execution (&& for success, || for failure)
- Logging errors to file

8.3 Try-Catch Equivalent

- No native try-catch (workaround using ERRORLEVEL checks)
- Pattern: Run command, check ERRORLEVEL, branch accordingly

8.4 Graceful Degradation

- Provide fallback options (if tool missing, use alternative)
- User-friendly error messages
- Clean exit (close files, reset state)

8.5 Logging

- Log to file (ECHO message >> log.txt)
- Timestamped logs
- Separate logs for errors vs info

PART 2: INTERMEDIATE TOPICS (SYSTEM ADMINISTRATION)

MODULE 9: NETWORK OPERATIONS

9.1 PING (Connectivity Testing)

- Basic ping (PING hostname)
- Options (-n, -t, -w)
- Parsing output (checking if host is up)

9.2 IPCONFIG (IP Configuration)

- Display IP info (IPCONFIG /ALL)
- Release/renew IP (IPCONFIG /RELEASE, IPCONFIG /RENEW)
- Flush DNS cache (IPCONFIG /FLUSHDNS)

9.3 NETSTAT (Network Statistics)

- Display active connections (NETSTAT -AN)
- Show listening ports (NETSTAT -AN | FIND "LISTENING")
- Process IDs (NETSTAT -ANO)

9.4 NET Commands

- NET USE (map network drives)
- NET USER (manage users)
- NET SHARE (manage shares)
- NET START/STOP (manage services)

9.5 NSLOOKUP (DNS Lookup)

- Query DNS records
- Find IP of hostname
- Reverse lookup

9.6 TRACERT (Trace Route)

- Trace packet path to destination
- Identify network bottlenecks

9.7 ARP (Address Resolution Protocol)

- Display ARP cache (ARP -A)
- Add static ARP entry
- Use cases (troubleshooting, network mapping)

9.8 CURL/CERTUTIL (Downloading Files)

- CURL (if available, Windows 10 1803+)
- CERTUTIL (download files via URL)
- PowerShell fallback (Invoke-WebRequest)

9.9 FTP Automation

- Scripting FTP commands
- Non-interactive FTP (FTP -s:script.txt)

MODULE 10: SYSTEM ADMINISTRATION

10.1 Task Scheduling (SCHTASKS)

- Create scheduled task (SCHTASKS /CREATE)
- Delete task (SCHTASKS /DELETE)
- Query tasks (SCHTASKS /QUERY)
- Modify task (SCHTASKS /CHANGE)

10.2 Service Management

- SC command (service control)
- NET START/STOP (start/stop services)
- Query service status (SC QUERY)
- Configure service startup type

10.3 Registry Operations (REG)

- Query registry (REG QUERY)
- Add registry key/value (REG ADD)
- Delete registry key/value (REG DELETE)
- Export/import registry (REG EXPORT, REG IMPORT)
- Use cases (persistence, configuration, forensics)

10.4 Process Management

- TASKLIST (list running processes)
- TASKKILL (kill process by name or PID)

- WMIC (Windows Management Instrumentation, advanced queries)

10.5 System Information

- SYSTEMINFO (display system details)
- WMIC queries (detailed hardware/software info)
- Parsing output (extracting specific fields)

10.6 Disk Operations

- DISKPART (disk partitioning, dangerous!)
- CHKDSK (check disk for errors)
- FORMAT (format drive)
- VOL (display volume label)

10.7 User and Group Management

- NET USER (add, delete, modify users)
- NET LOCALGROUP (manage groups)
- Listing users and groups

MODULE 11: ADVANCED TECHNIQUES

11.1 Arrays (Pseudo-Arrays)

- No native arrays (workaround using indexed variables)
- Creating array (SET arr[0]=value)
- Looping through array (FOR loop with counter)

11.2 Parsing JSON/XML

- No native parser (use external tools)
- Method 1: FOR /F with FINDSTR (regex parsing)
- Method 2: Call PowerShell from batch
- Method 3: Use jq.exe or xml.exe

11.3 Interacting with PowerShell

- Calling PowerShell from batch (PowerShell -Command "...")
- Passing variables to PowerShell

- Capturing PowerShell output in batch
- Hybrid batch/PowerShell scripts

11.4 Date and Time Manipulation

- Parsing %DATE% and %TIME%
- Extracting year, month, day, hour, minute, second
- Date arithmetic (add/subtract days, workaround)
- Formatting dates (YYYY-MM-DD, DD/MM/YYYY)

11.5 Obfuscation Techniques

- Variable indirection
- Encoding commands (Base64, ROT13)
- String concatenation obfuscation
- Use cases (bypassing AV, CTF challenges, educational only)

11.6 Self-Extracting Scripts

- Embedding files in batch script (Base64 encoded)
- Extracting at runtime (CERTUTIL -decode)
- Creating portable installers

11.7 GUI Elements (Basic)

- MSHTA (HTML Application, popup windows)
- VBScript integration (MsgBox, InputBox)
- PowerShell GUI (Windows Forms from batch)

PART 3: REAL-WORLD PROJECTS (PRACTICAL APPLICATIONS)

MODULE 12: Automation Projects

12.1 System Backup Script

- Backup files to external drive
- Incremental vs full backup
- Compression (using 7-Zip or tar)
- Scheduling via SCHTASKS

12.2 Log File Analyzer

- Parse log files (Apache, IIS, custom logs)
- Extract errors, warnings
- Generate summary report
- Email report (using SMTP via external tool)

12.3 Automated Software Installer

- Silent install multiple programs
- Check if already installed (registry check)
- Download installers (CURL/CERTUTIL)
- Configure post-install settings

12.4 Network Monitoring Script

- Ping multiple hosts
- Check service availability (HTTP, FTP, etc.)
- Log results with timestamp
- Alert on failure (email, log, sound)

12.5 File Organizer

- Sort files by extension (images, docs, videos)
- Move to categorized folders
- Handle duplicates (rename or skip)
- Dry-run mode (preview changes)

12.6 Batch File Menu System

- Display menu with numbered options
- User selects option
- Execute corresponding action
- Loop back to menu

12.7 Git Automation Script

- Automate git add, commit, push
- Daily backup to GitHub

- Branch management
- Error handling (check if git installed, repo exists)

12.8 Disk Cleanup Automation

- Delete temp files (%TEMP%, C:\Windows\Temp)
- Empty recycle bin
- Clear browser cache
- Schedule weekly

12.9 User Account Management

- Bulk create users (from CSV)
- Set passwords, groups
- Disable inactive accounts
- Generate report

12.10 Scheduled Maintenance Script

- Combine multiple tasks (backup, cleanup, updates)
- Run at specific time (SCHTASKS)
- Email summary report

PART 4: ETHICAL HACKING & MALWARE ANALYSIS (REDTEAM/SECURITY)

MODULE 13: Reconnaissance

13.1 Network Enumeration

- Discover active hosts (ping sweep)
- Port scanning (basic, no nmap equivalent in pure batch)
- ARP scanning (identify devices on local network)
- DNS enumeration (zone transfer attempts)

13.2 System Enumeration

- OS version (SYSTEMINFO)
- Installed software (WMIC, registry queries)
- Running processes (TASKLIST)

- Logged-in users (QUERY USER, NET SESSION)
- Network configuration (IPCONFIG, ROUTE PRINT)

13.3 User Enumeration

- List local users (NET USER)
- List domain users (NET USER /DOMAIN)
- Group membership (NET LOCALGROUP)
- Recently accessed files (recent folder, registry)

13.4 Credential Harvesting

- Stored credentials (CMDKEY /LIST)
- Browser passwords (external tools, NirSoft)
- Wi-Fi passwords (NETSH WLAN SHOW PROFILES, NETSH WLAN SHOW PROFILE name KEY=CLEAR)
- Registry-stored credentials

13.5 File Searching

- Search for sensitive files (passwords.txt, config files)
- Recursive directory search (FOR /R + FINDSTR)
- Keyword-based search (passwords, credentials, API keys)

MODULE 14: Persistence

14.1 Registry Run Keys

- Add to startup (HKCU\Software\Microsoft\Windows\CurrentVersion\Run)
- System-wide startup (HKLM version)
- Using REG ADD command

14.2 Scheduled Tasks

- Create task that runs at logon (SCHTASKS /CREATE)
- Hidden tasks (remove from Task Scheduler GUI)

14.3 Startup Folder

- Copy script to Startup folder (%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup)

14.4 Service Creation

- Create Windows service (SC CREATE)
- Auto-start service
- Disguising as legitimate service (naming)

14.5 DLL Hijacking (Batch as Dropper)

- Batch script drops malicious DLL
- Places in application directory
- Application loads DLL on next run

14.6 WMI Event Subscriptions

- Create WMI event filter (triggers on event)
- Execute batch script via WMI
- Stealthy persistence (hard to detect)

MODULE 15: Privilege Escalation

15.1 UAC Bypass Techniques

- Fodhelper.exe method (registry hijacking)
- Eventvwr.exe method (registry hijacking)
- Sdclt.exe method
- Disclaimer: Educational purposes only, illegal if used maliciously

15.2 Exploiting Weak Permissions

- Writable service binaries (replace with malicious batch)
- Writable startup folders
- DLL search order hijacking

15.3 Token Impersonation (Using External Tools)

- Batch script calls external tool (e.g., Incognito, custom C# binary)
- Elevate from user to SYSTEM

15.4 Scheduled Task Abuse

- Tasks running as SYSTEM or admin

- Modify task to run batch script

MODULE 16: Lateral Movement

16.1 PsExec-Like Functionality

- Batch script copies itself to remote machine
- Executes via WMI or scheduled task
- Requires admin credentials

16.2 WMI Remote Execution

- WMIC /NODE:"remote-pc" PROCESS CALL CREATE "batch-script.bat"
- Credential passing

16.3 SMB File Sharing

- Copy payload to network share
- Execute remotely (WMIC, SCHTASKS /S)

16.4 Pass-the-Hash (Using External Tools)

- Batch script orchestrates attack
- Calls Mimikatz or other tools

MODULE 17: Data Exfiltration

17.1 File Compression

- Compress sensitive files (7-Zip, tar)
- Password-protect archive

17.2 Uploading to External Server

- FTP upload
- HTTP POST (CURL)
- Cloud storage (Dropbox API via CURL)

17.3 Email Exfiltration

- Send file via email (SMTP using external tool)
- Encode file as Base64 (CERTUTIL)

17.4 DNS Exfiltration

- Encode data in DNS queries
- Extract via monitoring DNS server
- Advanced technique, stealthy

17.5 Steganography

- Hide data in image file (using external tool)
- Batch script automates embedding

MODULE 18: Covering Tracks

18.1 Clearing Event Logs

- WEVTUTIL command (clear specific logs)
- Clear all logs (dangerous, obvious)

18.2 Deleting Files Securely

- Simple delete (DEL, recoverable)
- Secure delete (overwrite with random data, SDelete tool)

18.3 Timestamp Manipulation

- Modify file timestamps (using PowerShell or external tool)
- Match original timestamps (before and after)

18.4 Disabling Logging

- Stop Windows Event Log service (risky, detectable)
- Modify audit policies

MODULE 19: Malware Analysis (Defensive)

19.1 Static Analysis

- Examine batch script without executing
- Extract IOCs (IPs, URLs, file paths)
- Deobfuscate (decode Base64, reverse string manipulation)

19.2 Dynamic Analysis (Sandboxing)

- Run script in VM
- Monitor file system changes (Process Monitor)
- Monitor registry changes
- Monitor network traffic (Wireshark)

19.3 Behavioral Analysis

- What does the script DO? (persistence, exfil, destruction)
- Identify malicious patterns

19.4 Signature Creation

- Create YARA rule for batch malware
- Identify unique strings, patterns

19.5 Remediation

- Remove persistence mechanisms (registry, scheduled tasks)
- Delete dropped files
- Restore modified settings

MODULE 20: Payload Development

20.1 Reverse Shell (Batch-Based)

- Connect back to attacker machine
- Execute commands remotely
- Using NETCAT or PowerShell

20.2 Keylogger (Batch as Dropper)

- Batch script downloads/executes keylogger
- Logs keystrokes to file
- Exfiltrates periodically

20.3 Ransomware Simulation (Educational)

- Encrypt files (using CIPHER or external tool)
- Display ransom note
- Decrypt on "payment" (password input)

- **Disclaimer: Illegal to deploy without consent, educational only**

20.4 Backdoor

- Persistent access mechanism
- Hidden communication channel
- Command execution via HTTP, DNS, etc.

20.5 Dropper/Downloader

- Batch script downloads second-stage payload
- Executes payload
- Deletes itself (cover tracks)

MODULE 21: Anti-Forensics

21.1 Fileless Malware Techniques

- Execute PowerShell in memory (no file written to disk)
- Load DLL reflectively

21.2 Living-Off-the-Land Binaries (LOLBins)

- Use legitimate Windows tools for malicious purposes
- Examples: CERTUTIL, MSHTA, REGSVR32, WMIC

21.3 Encoding and Obfuscation

- Base64 encoding (CERTUTIL)
- XOR encryption
- Variable name randomization
- String splitting and concatenation

21.4 Process Injection (Batch as Orchestrator)

- Batch script calls external tool (injects DLL into process)
- Evades detection

MODULE 22: Red Team Operations

22.1 Phishing Payloads

- Batch script disguised as legitimate file (document.bat.exe)
- Social engineering (naming, icons)

22.2 USB Drop Attacks

- Autorun batch script on USB insert
- Payloads (reverse shell, keylogger, exfiltrator)

22.3 Physical Access Attacks

- Bootable USB with batch scripts
- Modify startup scripts (if physical access)

22.4 Post-Exploitation Framework Integration

- Batch script as stage-1 (downloads Metasploit/Cobalt Strike beacon)
- Executes in memory

MODULE 23: Defensive Batch Scripting

23.1 Security Hardening Scripts

- Disable unnecessary services
- Configure Windows Firewall
- Apply security policies (via registry or GPO)

23.2 Intrusion Detection

- Monitor critical files for changes (hash comparison)
- Alert on unauthorized access attempts
- Log anomalies

23.3 Automated Incident Response

- Isolate infected machine (disable network)
- Kill malicious processes
- Collect forensic artifacts (logs, memory dumps)

23.4 Compliance Auditing

- Check system configuration against baseline
- Generate compliance report

- Remediate non-compliant settings

FINAL MODULE 24: Best Practices and Resources

24.1 Code Organization

- Modular design (functions for reusability)
- Meaningful variable names
- Consistent indentation and formatting

24.2 Security Considerations

- Never hardcode passwords (use encrypted storage or prompt)
- Validate user input (prevent injection attacks)
- Least privilege principle (don't run as admin unless necessary)

24.3 Performance Optimization

- Minimize loops (expensive operations)
- Use GOTO sparingly (spaghetti code risk)
- Leverage external tools when batch is inefficient

24.4 Cross-Version Compatibility

- Test on Windows 7, 10, 11, Server versions
- Use conditional logic for OS-specific features

24.5 Documentation and Commenting

- Document each function's purpose
- Explain non-obvious logic
- Include usage examples

24.6 Resources

- SS64.com (comprehensive batch reference)
- DosTips.com (community forum, advanced techniques)
- GitHub repositories (batch script examples)
- Windows Command-Line Reference (Microsoft Docs)